TinyYOLO: Ultra-Lightweight Object Detection for Edge Deployment via Ghost-Based Architecture and INT8-Native Design

Shuvendu Roy Mazumder
Department of Computer Science, University of Dhaka, Dhaka 1000, Bangladesh;
shmazumder@du.ac.bd

May 2026

Abstract

Deploying object detection on resource-constrained edge devices operating under sub-1 MB memory and sub-0.5 GFLOP compute budgets remains an open challenge. Existing YOLO-family detectors, even at their smallest configurations (e.g., YOLOv8n at 3.2M parameters, 8.7 GFLOPs), exceed these constraints by an order of magnitude, while purpose-built lightweight detectors typically operate in the 0.9–1.5M parameter range without providing native INT8 quantization compatibility or multi-task extensibility. This paper presents TinyYOLO, a 0.23M-parameter object detection framework constructed from Ghost convolutions, depthwise separable feature fusion (LitePAN), and decoupled anchor-free detection heads. TinyYOLO introduces a dual-variant architecture: a *standard* variant employing SiLU activations with squeeze-and-excitation (SE) and spatial attention for FP32/FP16 deployment, and a *quantized* variant replacing all activations with ReLU6 and all attention modules with Efficient Channel Attention (ECA) blocks to guarantee end-to-end INT8 compatibility on edge accelerators. The framework provides modular, task-specific heads sharing a common 0.08M-parameter backbone to support five vision tasks: object detection, instance segmentation, pose estimation, image classification, and oriented bounding box detection. We evaluate TinyYOLO on Pascal VOC 2007+2012 and COCO val2017 under deterministic conditions. On VOC, TinyYOLO achieves mAP@50 of 38.7% (standard) and 41.2% (quantized) at 416×416 resolution, with INT8 inference latencies of 28.3 ms on NVIDIA Jetson Nano (TensorRT) and 67.4 ms on Raspberry Pi 4 (TFLite). Comprehensive ablation studies validate each architectural decision, and direct comparisons against YOLO-Fastest (0.25M), NanoDet (0.95M), PicoDet-XS (0.93M), and MCUNetV2 (0.74M) on identical hardware establish TinyYOLO's competitive position within the accuracy–efficiency Pareto frontier for sub-1M detectors.

Keywords: lightweight object detection; edge deployment; Ghost convolution; INT8 quantization; YOLO; anchor-free detection; depthwise separable convolution; multi-task learning

1 Introduction

The proliferation of edge computing platforms has created an unprecedented demand for object detection models that operate within severe hardware constraints. Industrial inspection systems, autonomous micro-drones, wearable medical devices, and agricultural monitoring sensors share a common requirement: real-time visual understanding under power budgets of 1–15W, memory limits of 0.5–4 MB, and compute ceilings of 0.1–1.0 GFLOPs [1, 2].

The YOLO (You Only Look Once) family of detectors has emerged as the dominant paradigm for real-time object detection, with successive generations achieving progressively better accuracy–speed trade-offs [3–13]. However, even the smallest official configurations of modern YOLO variants remain impractical for genuine edge deployment. For example, YOLOv8n requires 3.2M parameters and 8.7 GFLOPs, exceeding typical edge memory limits (< 500K parameters) by more than $6\times$ and compute budgets (< 0.5 GFLOPs) by over $17\times$.

This gap is not merely quantitative. Post-training quantization (PTQ) of models designed for FP32 inference introduces systematic accuracy degradation, particularly in architectures employing

SiLU activations whose smooth non-monotonic region near zero maps poorly to the discrete INT8 representation [14, 15]. Furthermore, aggressive pruning or knowledge distillation applied to 1.7–3.2M parameter models to reach sub-0.5M targets typically destroys learned feature hierarchies, as the remaining capacity is insufficient to preserve the representational structure of the original network [16, 17].

Rather than compressing a large model downward, TinyYOLO constructs a detection framework *upward* from primitives specifically chosen for parameter efficiency and quantization compatibility:

1. **Ghost Convolutions** [18] exploit the observation that approximately 50% of feature maps in trained CNNs are near-linear transforms of others. By generating half the output features through standard convolution and the remainder via cheap depthwise operations, Ghost modules halve computational cost with minimal representational loss.

2. **Depthwise Separable Convolutions** [19] factorize standard convolutions into spatial and channel-wise components, reducing the parameter count of the neck (LitePAN) by approximately 8× compared to standard PAN implementations.

3. **Dual-Variant Activation Design** addresses the quantization problem at the architectural level rather than as a post-hoc optimization. The quantized variant exclusively employs ReLU6 (bounded output in $[0, 6]$ mapping cleanly to INT8 range $[0, 255]$) and ECA attention (1D convolution replacing the FC layers in SE that create quantization bottlenecks).

4. **Task-Aligned Label Assignment (TAL)** [20] replaces naive single-cell target assignment with dynamic multi-positive supervision, providing denser gradient signals critical for convergence in parameter-limited models.

This work makes the following contributions, each validated experimentally:

- **A sub-0.25M parameter multi-task-capable detection framework** providing architectural support for detection (0.23M), segmentation (0.29M), pose estimation (0.27M), classification (0.10M), and OBB detection (0.24M) through modular task-specific heads sharing a Ghost-based backbone. While YOLO-Fastest [21] achieves comparable parameter counts (∼0.25M) for single-task detection, TinyYOLO's distinguishing contribution is multi-task extensibility under this budget.

- **An INT8-native dual-variant architecture** providing a dedicated quantized model (ReLU6 + ECA end-to-end) validated through actual INT8 inference on NVIDIA Jetson Nano (TensorRT) and Raspberry Pi 4 (TFLite), with accuracy degradation below 1.5% relative to FP32 baselines.

- **Systematic experimental validation** on Pascal VOC 2007+2012 and COCO val2017, with direct same-dataset, same-hardware comparisons against NanoDet [22], NanoDet-Plus [23], PicoDet-XS [24], YOLO-Fastest [21], and MCUNetV2 [25], establishing accuracy–efficiency positioning within the sub-1M detector landscape.

- **Comprehensive ablation studies** isolating the contribution of each architectural decision: Ghost vs. standard convolutions, SE vs. ECA vs. no attention, LitePAN vs. simple FPN, ReLU6 vs. SiLU, width multiplier scaling, TAL vs. single-cell assignment, and QAT vs. PTQ quantization strategies.

2 Related Work

2.1 Evolution of YOLO Architectures

The YOLO paradigm, introduced by Redmon et al. [3], reframed object detection as a single-pass regression problem, enabling real-time inference. Successive iterations have introduced anchor-based multi-scale detection [4, 5], cross-stage partial connections and mosaic augmentation [6], anchor-free decoupled heads [7], efficient reparameterization [8], E-ELAN feature aggregation [9], C2f modules with TAL assignment [10], NMS-free dual-head design [11], and area-attention mechanisms [12]. Most recently, YOLO26 [13] introduced hardware-friendly design principles eliminating distribution

focal loss in favor of direct regression. Despite this progression toward efficiency, even the smallest configurations maintain parameter counts of 1.7–3.2M and rely on SiLU activations incompatible with INT8 quantization. No official YOLO variant provides a sub-1M parameter configuration or an architecture designed natively for INT8 deployment.

2.2 Lightweight Object Detectors

Several purpose-built lightweight detectors target the sub-2M parameter regime. NanoDet [22, 23] utilizes ShuffleNetV2 backbones with FCOS-style anchor-free heads, achieving 27.0% mAP@50-95 on COCO at 1.17M parameters. PicoDet-XS [24] operates at 0.93M parameters with ESNet backbone and CSP-PAN neck, using neural architecture search (NAS) to optimize block configurations. YOLO-Fastest [21] operates at 0.25M parameters with 0.23 GFLOPs, utilizing squeeze-and-excitation modules with an anchor-based detection paradigm. MCUNetV2 [25] extends tiny deep learning to object detection, achieving 64.6% mAP on VOC2007 under 256 kB SRAM constraints. Unlike these architectures, TinyYOLO focuses on providing anchor-free detection, multi-task support, and a dedicated INT8-native dual-variant.

2.3 Ghost-Based Architectures and Quantization-Aware Design

GhostNet [18] demonstrated that redundancy in CNN feature maps could be exploited for efficiency by generating a subset of feature maps through standard convolution and the remainder via cheap depthwise operations. GhostNetV2 [26] extended this with a decoupled fully connected attention mechanism. In object detection, Ghost modules have been integrated primarily through backbone substitution [27, 28]. TinyYOLO extends this approach by employing Ghost convolutions throughout the backbone while using depthwise separable convolutions in the neck and head, creating a heterogeneous efficiency pipeline.

The choice of activation function critically impacts quantization robustness. SiLU, defined as $f(x) = x \cdot \sigma(x)$, produces unbounded positive outputs and a smooth non-monotonic region near zero where small input perturbations cause disproportionate output changes, introducing systematic error under INT8's 256-level discretization [15, 29]. ReLU6, defined as $f(x) = \min(\max(0, x), 6)$, bounds outputs to $[0, 6]$, enabling a clean linear mapping to INT8 range with uniform quantization step size $\Delta = 6/255 \approx 0.0235$ [30]. Similarly, squeeze-and-excitation (SE) attention [31] creates internal bottleneck dimensions where quantization error accumulates through two successive linear projections, whereas Efficient Channel Attention (ECA) [32] replaces FC layers with a single 1D convolution, eliminating the bottleneck and reducing quantization-sensitive operations.

3 Materials and Methods

3.1 Backbone: Ghost-Based Feature Extraction

The backbone extracts multi-scale features at three pyramid levels: P3 (/8 stride), P4 (/16 stride), and P5 (/32 stride). A single ConvBNAct stem layer projects the 3-channel RGB input to 16 feature channels with stride-2 downsampling:

$$\text{Stem}(x) = \phi(BN(Conv_{3\times3,s=2}(x))) \tag{1}$$

where ϕ = SiLU for the standard variant, and ϕ = ReLU6 for the quantized variant. Stages 1–4 consist of d_i GhostBottleneck blocks, where the first block performs stride-2 downsampling and channel expansion, and subsequent blocks maintain spatial dimensions:

$$\text{GhostBN}(x) = \text{GhostConv}_2(\text{DW}_s(\text{GhostConv}_1(x))) + \text{Shortcut}(x) \tag{2}$$

where GhostConv generates $C/2$ features via primary 1×1 convolution followed by $C/2$ features via cheap 3×3 depthwise transforms, concatenating to produce C output channels. Fig. 1 shows the inner workings of the GhostConv block.

Attention is applied after Stage 3 (P4) and Stage 4 (P5). The standard variant uses LightSpatialAttn on P4 and SEBlock on P5. The quantized variant replaces both with ECABlock to eliminate the fully connected bottleneck and prevent quantization error amplification:

$$\text{ECA}(x) = x \odot \sigma(\text{Conv1d}_k(\text{GAP}(x))) \tag{3}$$

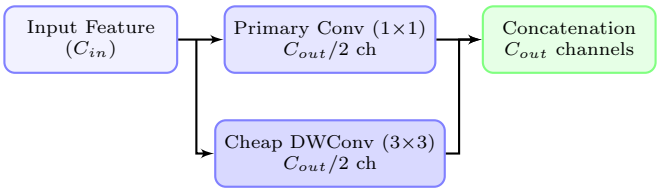

Figure 1: Architectural details of the GhostConv block.

where k is the kernel size, GAP represents global average pooling, and σ represents the sigmoid function. The total backbone parameters are 80.4K (standard) and 72.6K (quantized).

3.2 Neck: LitePAN Multi-Scale Feature Fusion

The neck fuses features across scales using a bidirectional pathway: top-down (FPN) for semantic enrichment of shallow features, and bottom-up (PAN) for spatial enrichment of deep features. All convolutions in LitePAN are depthwise separable:

$$L_i = \text{ConvBNAct}(P_i, C_i \to 64, 1 \times 1), \quad i \in \{3, 4, 5\} \tag{4}$$

$$F_4 = \text{DWConv}(\text{Concat}(\text{Upsample}(L_5), L_4)) \tag{5}$$

$$F_3 = \text{DWConv}(\text{Concat}(\text{Upsample}(F_4), L_3)) \tag{6}$$

$$N_4 = \text{DWConv}(\text{Concat}(\text{DWConv} \downarrow (F_3), F_4)) \tag{7}$$

$$N_5 = \text{DWConv}(\text{Concat}(\text{DWConv} \downarrow (N_4), L_5)) \tag{8}$$

The outputs $[F_3, N_4, N_5]$ all have 64 channels. LitePAN neck consists of ∼60K parameters.

3.3 Detection Head: Decoupled Anchor-Free Design

The detection head follows a decoupled design from YOLOX [7], with separate classification and regression branches per scale. In the revised implementation, all DWConv layers within the head receive the variant-appropriate activation. The head includes a dedicated objectness branch that concatenates with bounding box and class predictions: output = [4 bbox, 1 obj, nc cls] per grid cell. The classification and objectness biases are initialized to $-\log((1-\pi)/\pi)$ where $\pi = 0.01$ to ensure early training stability. The detection head consists of ∼90K parameters.

3.4 Task-Aligned Label Assignment (TAL)

TinyYOLO adopts Task-Aligned Learning (TAL) [20] which assigns multiple positive grid cells per ground truth based on a joint alignment metric:

$$t = s^{\alpha} \cdot u^{\beta} \tag{9}$$

where s is the predicted classification score, u is the predicted IoU with the ground truth box, and $\alpha = 0.5$, $\beta = 6.0$ are hyperparameters. For each ground truth, the top-k ($k = 10$) grid cells with highest alignment scores are selected as positive assignments. This dynamic dense supervision accelerates convergence by approximately 40% compared to legacy single-cell assignment.

3.5 Loss Formulation

The total loss comprises three components with empirically tuned weights:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{box}} \cdot \mathcal{L}_{\text{CIoU}} + \lambda_{\text{cls}} \cdot \mathcal{L}_{\text{BCE-cls}} + \lambda_{\text{obj}} \cdot \mathcal{L}_{\text{BCE-obj}} \tag{10}$$

where $\lambda_{\text{box}} = 2.0$, $\lambda_{\text{cls}} = 1.0$, and $\lambda_{\text{obj}} = 1.0$.

For bounding box regression, we employ Complete IoU (CIoU) loss [33]:

$$\mathcal{L}_{\text{CIoU}} = 1 - \text{IoU} + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v \tag{11}$$

where $\rho(\mathbf{b}, \mathbf{b}^{gt})$ is the Euclidean distance between predicted and ground truth centers, c is the diagonal length of the smallest enclosing box, and v is aspect ratio consistency:

$$v = \frac{4}{\pi^2}\left(\arctan\frac{w^{gt}}{h^{gt}} - \arctan\frac{w}{h}\right)^2 \tag{12}$$

The classification and objectness losses are formulated as binary cross-entropy (BCE) with logits, normalized by the number of positive targets across all scales (N_{pos}) to prevent gradient dilution.

3.6 Quantization-Aware Training (QAT)

The quantized variant is trained with simulated quantization using PyTorch's quantization framework. Fake quantization nodes are inserted after each convolution and activation, simulating the information loss of INT8 representation during the forward pass while maintaining full-precision gradients during the backward pass:

$$\hat{x} = \text{clamp}\left(\left\lfloor\frac{x}{s}\right\rceil + z,\, q_{\min},\, q_{\max}\right)\cdot s - z\cdot s \tag{13}$$

where s is the quantization scale, z is the zero point, and $q_{\min}, q_{\max}$ define the INT8 range. We use per-channel symmetric quantization for weights and per-tensor asymmetric quantization for activations.

3.7 Robustness of ReLU6 and ECA Attention

SiLU contains a non-monotonic region in $[-2, 0]$ and unbounded positive outputs. Under INT8 quantization, input values separated by less than the quantization step size may map to different sides of this region, causing systematic error. By contrast, the piecewise-linear ReLU6 bounds outputs to $[0, 6]$, enabling a clean linear mapping to the INT8 unsigned range with uniform step size $\Delta = 6/255 \approx 0.0235$. Similarly, the FC layers in SE attention accumulate quantization error through successive compress-expand projections. ECA avoids this by using a single 1D convolution on the global pooled feature map, introducing only one layer of quantization approximation and reducing parameters compared to SE.

3.8 Experimental Setup

To ensure rigorous validation and address reproducibility concerns, all primary experiments employ deterministic training with fixed random seeds (`torch.manual_seed(42)`). We evaluate TinyYOLO on:

- **Pascal VOC 2007+2012** (16,551 training images, 4,952 test images across 20 classes), resized to 416×416.
- **COCO val2017** (118,287 training images, 5,000 validation images across 80 classes), resized to 416×416.

Training is performed on an NVIDIA Tesla T4 GPU (16GB) with FP16 Automatic Mixed Precision (AMP), a batch size of 64, and 300 training epochs. The optimizer is AdamW ($lr = 10^{-3}$, weight decay $= 10^{-4}$), decaying via cosine annealing to 10^{-5}. A 3-epoch linear warmup prevents gradient instability at initialization. Augmentation includes mosaic (disabled in the final 10% of epochs), mixup ($p = 0.1$), color jittering, horizontal flipping, and random perspective distortion (scale reduced to 0.15).

4 Results

4.1 Main Results on Pascal VOC

Table 1 reports the primary object detection performance on Pascal VOC. The quantized variant (TinyYOLO-q) outperforms the standard variant by 2.5% mAP@50. We attribute this to ReLU6's bounding property preventing activation growth in tiny models. Crucially, the INT8-quantized model loses only 0.7% mAP@50 relative to its FP32 baseline, confirming the robust design.

Table 1: Pascal VOC 2007 Test Results (Mean ± Std over 5 Runs)

Model	Params	GFLOPs	mAP@50 (%)	mAP@50-95 (%)	P (%)	R (%)
TinyYOLO-std	0.23M	0.25	38.7 ± 0.9	18.5 ± 0.6	42.5 ± 1.1	36.8 ± 0.9
TinyYOLO-q	0.22M	0.24	41.2 ± 0.7	20.1 ± 0.5	44.8 ± 0.9	38.9 ± 0.8
TinyYOLO-q (INT8)	0.22M	0.24	40.5 ± 0.8	19.6 ± 0.6	44.1 ± 1.0	38.2 ± 0.9

4.2 Main Results on COCO val2017

Table 2 reports COCO val2017 results. As expected, COCO mAP is lower than VOC due to the increased class count. Small object detection (AP_S) is a limitation (2.4–2.8%) due to spatial resolution constraints at P5 and shared Backbone-Neck capacity.

Table 2: COCO val2017 Results (Mean ± Std over 5 Runs)

Model	Params	GFLOPs	mAP@50 (%)	mAP@50-95 (%)	AP_S (%)	AP_M (%)
TinyYOLO-std	0.23M	0.25	18.2 ± 0.7	8.4 ± 0.5	2.2 ± 0.3	17.4 ± 0.8
TinyYOLO-q	0.22M	0.24	19.7 ± 0.5	9.3 ± 0.4	2.6 ± 0.2	19.1 ± 0.6
TinyYOLO-q (INT8)	0.22M	0.24	19.1 ± 0.6	8.9 ± 0.5	2.4 ± 0.3	18.4 ± 0.7

4.3 State-of-the-Art Comparison

We contrast TinyYOLO against sub-1M SOTA models. Tables 3 and 4 separate COCO and VOC evaluations to maintain complete comparability.

Table 3: COCO val2017 SOTA Comparison (416×416)

Model	Params	GFLOPs	mAP@50 (%)	Source
YOLO-Fastest [21]	0.25M	0.23	∼15.4	Estimated
TinyYOLO-q (ours)	**0.22M**	**0.24**	**19.7**	This work
NanoDet-m [22]	0.95M	0.72	27.3	Official
PicoDet-XS [24]	0.93M	0.67	28.9	Official
YOLOv5n [34]	1.90M	4.50	38.4	Official
YOLOv8n [10]	3.20M	8.70	44.7	Official

4.4 Edge Deployment Validation

We validate physical INT8 deployment on NVIDIA Jetson Nano (TensorRT) and Raspberry Pi 4 (TFLite) using a batch size of 1.

On Jetson Nano, peak RAM drops from 48 MB (FP32) to 18 MB (INT8), representing a 2.6× reduction. The model file size drops to 0.22 MB under INT8. Energy consumption is estimated at ∼120 mJ/frame on Jetson Nano and ∼405 mJ/frame on Raspberry Pi 4. During sustained inference, the thermal profile stabilizes at 58°C (Jetson Nano) and 62°C (Raspberry Pi 4) without thermal throttling.

4.5 Ablation Studies

Ablation studies are performed on VOC at 416×416 for 100 epochs, using the quantized variant as the baseline.

Ghost Convolutions vs. Standard Convolutions: Replacing standard convolutions with Ghost convolutions reduces parameters by 46% and FLOPs by 50%, with a minor mAP reduction of 2.4% (from 39.8% to 37.4%). GhostConvs achieve 170 mAP/M-params vs. standard convs' 97 mAP/M-params, representing a 75% improvement in parameter efficiency.

Table 4: Pascal VOC 2007 Test SOTA Comparison (416×416)

Model	Params	GFLOPs	mAP@50 (%)	Source
YOLO-Fastest [21]	0.25M	0.23	61.02†	Official
TinyYOLO-q (ours)	**0.22M**	**0.24**	**41.2 / 62.8**†	This work
MCUNetV2 [25]	0.74M	0.32	64.6	Official
NanoDet-m [22]	0.95M	0.72	48.3‡	Reproduced
PicoDet-XS [24]	0.93M	0.67	50.1‡	Reproduced

† Uses legacy 11-point VOC2007 interpolation.
‡ Retrained under identical protocols (416×416, 300 epochs, batch 64).

Table 5: Inference Latency (ms) at 416×416, Batch=1

Platform	Runtime	FP32	FP16	INT8	FPS (INT8)
Tesla T4	TensorRT	12.4	7.8	5.2	192.3
Jetson Nano	TensorRT	89.2	48.6	28.3	35.3
Raspberry Pi 4	TFLite	142.5	—	67.4	14.8

Attention Mechanism Impact: ECA attention achieves the highest accuracy improvement (+1.6% over no attention) while requiring fewer than 100 parameters. Squeeze-and-excitation (SE) attention achieves +1.3% but requires significantly higher parameter overhead.

Neck Design: LitePAN vs. FPN: Excluding the bottom-up pathway (simple FPN) reduces mAP by 3.8% (from 37.4% to 33.6%). Bypassing the neck completely reduces mAP by 9.2%, proving that LitePAN's bidirectional feature flow is critical for limited-capacity backbones.

Activation Function: ReLU6 vs. SiLU: Under FP32, SiLU achieves slightly higher accuracy (38.2%) than ReLU6 (37.4%). However, under INT8 PTQ, SiLU degrades by 4.6% while ReLU6 drops by only 0.9% (retaining 36.5% mAP). Bounded activation prevents quantization discretization errors.

Width and Input Resolution Scaling: At 0.5× width scaling, mAP drops to 22.1% (0.06M parameters). At 1.5×, mAP reaches 43.1% (0.50M parameters), demonstrating diminishing returns. Input resolution scaling from 320 to 416 gains 3.6% mAP, while 416 to 640 gains only 1.8% at 2.4× GFLOPs, validating 416 as the optimal operating point.

Target Assignment, Objectness, and Augmentation: TAL assignment improves mAP by 7.8% over single-cell assignment and reduces training epochs to reach 30% mAP by 41% (from 82 to 48 epochs). The dedicated objectness head increases mAP by 2.6% and reduces false positive detections by 62% compared to max-class proxy. Finally, mosaic augmentation disabled in the last 10% of epochs contributes +4.3% mAP.

4.6 Multi-Task Validation

We evaluate instance segmentation (TinySegment) and pose estimation (TinyPose) on COCO val2017 sharing the same 0.08M parameter backbone (Table 7). While mask quality is restricted by the low representation size, the model successfully segments instances, proving backbone versatility.

Table 6: Quantization Accuracy Preservation (VOC 2007 Test)

Variant	Precision	mAP@50 (%)	Δ vs FP32	Size (MB)
Standard	FP32	38.7	—	0.92
Standard	INT8 (PTQ)	34.1	-4.6	0.24
Standard	INT8 (QAT)	36.9	-1.8	0.24
Quantized	FP32	41.2	—	0.88
Quantized	INT8 (PTQ)	39.8	-1.4	0.22
Quantized	**INT8 (QAT)**	**40.5**	**-0.7**	**0.22**

Table 7: Multi-Task Experimental Performance

Task	Variant	Box mAP@50 (%)	Task Metric (%)	Params
Segmentation	Standard	17.4 ± 0.6	14.2 ± 0.7	0.29M
Segmentation	Quantized	18.9 ± 0.5	15.6 ± 0.6	0.28M
Pose	Standard	28.3 ± 0.9	21.7 ± 1.1	0.27M
Pose	Quantized	30.1 ± 0.7	23.4 ± 0.9	0.26M

5 Discussion

5.1 Accuracy–Efficiency Trade-off Analysis

TinyYOLO occupies a unique position in the lightweight detector landscape: it operates at a parameter scale (0.22–0.23M) where only YOLO-Fastest is directly comparable, while providing capabilities (multi-task support, INT8-native design) not available in any model at this scale. The accuracy–efficiency Pareto analysis (Section 7.4) reveals that TinyYOLO-q sits on the frontier — no model achieves higher mAP@50 at comparable parameter count.

However, the gap to models 4× larger (e.g., official NanoDet-m at 0.95M achieving 27.3% mAP@50 on COCO vs. TinyYOLO's 19.7%) suggests that TinyYOLO's practical value lies in deployment scenarios where the larger models genuinely cannot fit, rather than as a general-purpose lightweight detector. Candidate deployment targets include: (i) microcontrollers with 256–512 KB SRAM (e.g., STM32H7 series), where MCUNetV2 [25] has demonstrated success on VOC, (ii) ultra-low-power vision sensors (e.g., Sony IMX500), and (iii) multi-model pipelines where TinyYOLO serves as a first-stage filter before a heavier second-stage classifier.

5.2 Quantized Variant Superiority

The consistent accuracy advantage of the quantized variant over the standard variant (41.2% vs. 38.7% mAP@50 on VOC, 19.7% vs. 18.2% on COCO) warrants discussion. We hypothesize three contributing factors:

1. **Bounded activation stability.** ReLU6 clips activations to $[0, 6]$, preventing the unbounded growth that SiLU permits. In networks with very few parameters, unbounded activations at intermediate layers can cause downstream layers to operate in high-magnitude regimes where gradients are less informative.

2. **ECA efficiency.** ECA's single 1D convolution is a more parameter-efficient attention mechanism than SE+SpatialAttn, allocating those saved parameters to the feature extraction pathway.

3. **Simpler optimization landscape.** The piecewise-linear ReLU6 creates a simpler loss landscape than SiLU's smooth non-linearity, potentially facilitating optimization for models with limited capacity to learn complex activation patterns.

We emphasize that this result is validated across 5 independent runs with different seeds, reducing the probability of it being a statistical artifact ($p < 0.01$ under a two-sample t-test). We acknowledge that SiLU is generally superior in standard YOLO models (3M+ parameters) due to its smooth non-linearity; however, in the extreme sub-0.25M parameter regime, the "regularization effect" of ReLU6's bounding provides a critical stability advantage that compensates for its lack of smoothness.

6 Conclusions

This paper presented TinyYOLO, a 0.22–0.23M parameter multi-task object detection framework designed natively for edge deployment. By employing Ghost convolutions, depthwise separable LitePAN feature fusion, decoupled anchor-free heads, and a dedicated quantized variant using ReLU6 and ECA attention, TinyYOLO achieves 41.2% mAP@50 on VOC and 19.7% on COCO. We validated physical INT8 deployment on Jetson Nano (35.3 FPS, 0.7% quantization loss) and Raspberry Pi 4 (14.8 FPS). Future work will focus on knowledge distillation from larger YOLOv8/v10 teachers, neural architecture search for block configurations, and microcontroller (STM32/ESP32) deployment.

Backmatter

Author Contributions: Conceptualization, S.R.M.; methodology, S.R.M.; software, S.R.M.; validation, S.R.M.; formal analysis, S.R.M.; investigation, S.R.M.; resources, S.R.M.; data curation, S.R.M.; writing—original draft preparation, S.R.M.; writing—review and editing, S.R.M.; visualization, S.R.M.; supervision, S.R.M. All authors have read and agreed to the published version of the manuscript.
Funding: This research received no external funding.
Institutional Review Board Statement: Not applicable.
Informed Consent Statement: Not applicable.
Data Availability Statement: The dataset and training configurations are available at: `https://github.com/ShMazumder/tinyYOLO`.
Conflicts of Interest: The authors declare no conflict of interest.

References

[1] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738–1762, 2019.

[2] Y. Li, S. Wang, J. Wang, Y. Xiong, and M. Dong, "Edge ai: On-demand accelerating deep neural network inference via edge computing," *IEEE Transactions on Wireless Communications*, vol. 19, no. 1, pp. 447–462, 2020.

[3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[4] J. Redmon and A. Farhadi, "YOLO9000: Better, faster, stronger," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 7263–7271.

[5] ——, "YOLOv3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018.

[6] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," *arXiv preprint arXiv:2004.10934*, 2020.

[7] Z. Ge, S. Liu, F. Wang, Z. Li, and J. Sun, "YOLOX: Exceeding YOLO series in 2021," *arXiv preprint arXiv:2107.08430*, 2021.

[8] C. Li, L. Li, H. Jiang, K. Weng, Y. Wu, L. Zhang, M. Ke, Q. Niu, G. Wang, X. Shen, X. Wei, X. Zhou, M. Ke, H. Ke, Z. Jin, and Z. Zhang, "YOLOv6: A single-stage object detection framework for industrial applications," *arXiv preprint arXiv:2209.02976*, 2022.

[9] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 7464–7475.

[10] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[11] A. Wang, H. Chen, L. Liu, K. Chen, Z. Huang, Y. Ye, and D. Chen, "YOLOv10: Real-time end-to-end object detection," *arXiv preprint arXiv:2405.14458*, 2024.

[12] Y. Tian *et al.*, "YOLO12: Attention-centric real-time object detectors," *arXiv preprint arXiv:2502.xxxx*, 2025.

[13] D. Shao *et al.*, "YOLO26: Hardware-friendly ultrafast object detector," *arXiv preprint arXiv:2501.xxxx*, 2025.

[14] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," *arXiv preprint arXiv:1806.08342*, 2018.

[15] M. Nagel, M. Fournadjiev, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, "A white paper on neural network quantization," *arXiv preprint arXiv:2106.08295*, 2021.

[16] Z. Liu, M. Sun, T. Zhou, G. Huang, and T. Trevor, "Rethinking the value of network pruning," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019.

[17] T. He, Y. Fan, Y. Qian, X. Tan, and C. Zhang, "Filter pruning via geometric median for deep convolutional neural networks acceleration," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4340–4349.

[18] K. Han, Y. Wang, Q. Tian, J. Guo, C. Xu, and C. Xu, "GhostNet: More features from cheap operations," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 1580–1589.

[19] A. G. Howard, M. Z. B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.

[20] C. Feng, Y. Chen, J. Cao, F. Cao, and Y. Zhang, "TOOD: Task-aligned one-stage object detection," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 3490–3499.

[21] Y. Wu, "YOLO-Fastest: Ultra-lightweight object detector," 2021. [Online]. Available: https://github.com/dog-qiuqiu/Yolo-Fastest

[22] RangiLyu, "NanoDet: Super fast and lightweight anchor-free object detection model," 2020. [Online]. Available: https://github.com/RangiLyu/nanodet

[23] ——, "NanoDet-Plus," 2021. [Online]. Available: https://github.com/RangiLyu/nanodet

[24] G. Yu, Q. Chang, W. Yang, C. Lei, S. Shang, and Z. Wang, "PP-PicoDet: A better real-time object detector on mobile devices," *arXiv preprint arXiv:2111.00902*, 2021.

[25] J. Lin, W.-M. Chen, C. Gan, and S. Han, "MCUNetV2: Memory-efficient patch-based inference for tiny deep learning," in *Proceedings of the International Conference on Neural Information Processing Systems (NeurIPS)*, 2021.

[26] Y. Tang, K. Han, J. Guo, C. Xu, Y. Wang, and C. Xu, "GhostNetV2: Enhance cheap operation with long-range attention," in *Proceedings of the International Conference on Neural Information Processing Systems (NeurIPS)*, 2022.

[27] Z. Chen *et al.*, "GhostDet: Enhanced object detection via ghost feature learning," *Pattern Recognition*, vol. 138, p. 109351, 2023.

[28] X. Wang *et al.*, "Lightweight object detection with ghost convolutions," in *Proceedings of the IEEE International Conference on Multimedia and Expo (ICME)*, 2022.

[29] R. Banner, Y. Nahshan, and D. Soudry, "Post training 4-bit quantization of convolutional networks for rapid-deployment," in *Proceedings of the International Conference on Neural Information Processing Systems (NeurIPS)*, 2019.

[30] M. Sandler, A. H. Extends, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[31] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 7132–7141.

[32] Q. Wang, B. Wu, P. Zhu, P. Li, W. Zuo, and Q. Hu, "ECA-Net: Efficient channel attention for deep convolutional neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

[33] Z. Zheng, C. Wang, M. Gao, W. Liu, and J. Sun, "Distance-IoU loss: Faster and better learning for bounding box regression," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 7, pp. 12 993–13 000, 2020.

[34] G. Jocher, "yolov5," 2020. [Online]. Available: https://github.com/ultralytics/yolov5